

Especificación de la API REST

Carlos Alarcón

13 de junio de 2026

1. Introducción

Este documento detalla la interfaz REST para el sistema **Polaflix**. La API ha sido refinada para exponer estrictamente los endpoints necesarios para satisfacer los requisitos del flujo principal de la aplicación: acceso de usuarios y visualización del catálogo de series.

Decisiones de Diseño y Optimización de Payloads

En el diseño de esta API se han aplicado las siguientes decisiones para optimizar el rendimiento y cumplir con los estándares REST:

- **Cuerpos Vacíos para Acciones de Estado:** Las operaciones que mutan el estado del sistema, como registrar una visualización (PUT), devuelven un código 200 OK sin cuerpo. Se delega la información del fallo a los códigos HTTP semánticos correspondientes (404 Not Found, 500 Internal Server Error).
- **Optimización de Carga (Prevención de Over-fetching):** Atendiendo a criterios de escalabilidad, el endpoint de recuperación del perfil del usuario excluye deliberadamente colecciones que tienden a crecer infinitamente (historial completo de **facturas** y el desglose total de **capitulosVistos**). Para el inicio de la aplicación, el **estadoSeries** es suficiente para construir las listas de interfaz, evitando penalizar los tiempos de carga en usuarios veteranos.
- **Gestión de Parámetros Mutuamente Exclusivos:** Las consultas genéricas implementan validaciones lógicas para impedir peticiones contradictorias (ej. enviar simultáneamente un filtro de inicial y uno de título), devolviendo 400 Bad Request.

Resumen de Endpoints Activos

- **Usuarios:**
 - GET /usuarios/{username} - Obtener perfil del usuario (Acceso).
 - PUT /usuarios/{username}/capitulos-vistos/{idCapitulo} - Registrar visualización.
- **Series:**
 - GET /series - Listar catálogo (con filtros opcionales).
 - GET /series/{id} - Obtener detalle completo de una serie.

2. Endpoints de Usuarios

2.1. Obtener Perfil de Usuario (Login / Acceso)

Endpoint: GET /usuarios/{username}

Recupera los datos de acceso del usuario, su plan de suscripción y el estado general del seguimiento de sus series. Para optimizar el tráfico de red, no devuelve el historial completo de facturas ni de capítulos.

Códigos de Estado HTTP

- **200 OK:** Petición exitosa. El usuario existe y se devuelven sus datos.
- **404 Not Found:** El nombre de usuario especificado no existe en el sistema.

Implementación en Java

```
@GetMapping("/{username}")
@JsonView(Views.UsuarioBasico.class)
@Operation(summary = "Entrar al sistema", description = "Comprueba si el
    usuario existe y devuelve sus datos.")
public ResponseEntity<Usuario> obtenerUsuario(@PathVariable("username") String
    username) {
    Usuario usuario = usuarioService.obtenerUsuario(username);
    if (usuario != null) {
        return ResponseEntity.ok(usuario);
    } else {
        return ResponseEntity.notFound().build();
    }
}
```

Ejemplo de Respuesta Exitoso (200 OK)

```
{
  "id": 15,
  "username": "cr7bicho",
  "iban": { "numeroCuenta": "ES0011112222333344445555" },
  "planSuscripcion": { "tarifaPlana": false, "cuotaMensual": 0.0 },
  "estadoSeries": {
    "1": "EMPEZADA",
    "3": "PENDIENTE"
  }
}
```

2.2. Registrar Visualización de Capítulo

Endpoint: PUT /usuarios/{username}/capitulos-vistos/{idCapitulo}

Añade un capítulo al historial del usuario. El sistema recalcula automáticamente el estado general de la serie (ej. transición a TERMINADA) y ejecuta la lógica de facturación si corresponde al modelo de negocio del usuario. No devuelve cuerpo de respuesta.

Códigos de Estado HTTP

- **200 OK:** Visualización registrada con éxito y lógica de dominio procesada.
- **404 Not Found:** El usuario no existe, o el ID del capítulo proporcionado es inválido.
- **500 Internal Server Error:** Error interno no controlado al intentar procesar la transacción o la facturación.

Implementación en Java

```
@PutMapping("/{username}/capitulos-vistos/{idCapitulo}")
@Operation(summary = "Marcar capítulo como visto", description = "Registra que
un usuario ha visto un capítulo.")
public ResponseEntity<Void> verCapitulo(
    @PathVariable("username") String username,
    @PathVariable("idCapitulo") Long idCapitulo) {
    try {
        usuarioService.verCapitulo(username, idCapitulo);
        return ResponseEntity.ok().build();
    } catch (java.util.NoSuchElementException e) {
        return ResponseEntity.notFound().build();
    } catch (Exception e) {
        return ResponseEntity.internalServerError().build();
    }
}
```

3. Endpoints de Series

3.1. Obtener Catálogo de Series

Endpoint: GET /series

Devuelve la colección de series disponibles en el sistema. Los atributos pesados (creadores, actores, temporadas) se omiten en esta vista. Dispone de dos parámetros de consulta (*query params*) **opcionales** para filtrado.

Parámetros de consulta (Query Params):

- **inicial** (String, Opcional): Devuelve las series cuya primera letra coincida.
- **titulo** (String, Opcional): Devuelve las series que contengan el texto indicado.

Nota: Ambos parámetros son mutuamente excluyentes. No se puede filtrar por inicial y título al mismo tiempo.

Códigos de Estado HTTP

- **200 OK:** Petición exitosa, se devuelve el array de series filtradas o completas.
- **204 No Content:** La petición fue correcta, pero la búsqueda no arrojó ningún resultado (el array está vacío).
- **400 Bad Request:** Error de uso. El cliente envió los parámetros inicial y titulo simultáneamente.

Implementación en Java

```
@GetMapping
@JsonView(Views.SerieResumida.class)
@Operation(summary = "Obtener catalogo", description = "Devuelve el catalogo de series.")
public ResponseEntity<List<Serie>> obtenerCatalogo(
    @RequestParam(required = false) String inicial,
    @RequestParam(required = false) String titulo) {

    boolean tieneInicial = inicial != null && !inicial.trim().isEmpty();
    boolean tieneTitulo = titulo != null && !titulo.trim().isEmpty();

    if (tieneInicial && tieneTitulo) {
        return ResponseEntity.badRequest().build();
    }

    List<Serie> catalogo = serieService.obtenerSeries(inicial, titulo);

    if (!catalogo.isEmpty()) {
        return ResponseEntity.ok(catalogo);
    } else {
        return ResponseEntity.noContent().build();
    }
}
```

Ejemplo de Respuesta Exitosa (200 OK)

```
[
  {
    "id": 1,
    "titulo": "Peaky Blinders",
```

```

    "sinopsis": "Una familia de gansteres asienta su dominio...",
    "costeVisionado": 1.5,
    "categoria": "GOLD"
  },
  {
    "id": 2,
    "titulo": "Prison Break",
    "sinopsis": "Michael Scofield elabora un plan magistral...",
    "costeVisionado": 0.75,
    "categoria": "SILVER"
  }
]

```

3.2. Obtener Detalle de Serie

Endpoint: GET /series/{id}

Recupera el grafo completo de una serie mediante su identificador numérico, lo que incluye a todos sus creadores, actores participantes, temporadas, y el listado íntegro de capítulos de cada temporada.

Códigos de Estado HTTP

- **200 OK:** Petición exitosa. Se devuelve el objeto JSON con todo el detalle de la serie.
- **404 Not Found:** No existe ninguna serie asociada al identificador proporcionado.

Implementación en Java

```

@GetMapping("/{id}")
@JsonView(Views.SerieDetallada.class)
@Operation(summary = "Obtener detalle de serie", description = "Devuelve los
    detalles completos de una serie, incluyendo temporadas, capitulos y actores.
")
public ResponseEntity<Serie> obtenerDetalleSerie(@PathVariable("id") Integer id
) {
    Serie serie = serieService.obtenerSeriePorId(id);

    if (serie != null) {
        return ResponseEntity.ok(serie);
    } else {
        return ResponseEntity.notFound().build();
    }
}

```

Ejemplo de Respuesta Exitosa (200 OK)

```

{
  "id": 3,
  "titulo": "La que se avecina",
  "sinopsis": "Las disparatadas vidas y problemas cotidianos...",
  "creadores": [
    { "id": 18, "nombre": "Alberto Caballero" },
    { "id": 19, "nombre": "Laura Caballero" }
  ],
  "actores": [
    { "id": 20, "nombre": "Pablo Chiapella" },
    { "id": 21, "nombre": "Jordi Sanchez" }
  ],
  "temporadas": [

```

```
{
  "id": 114,
  "titulo": "Temporada 1 de La que se avecina",
  "numero": 1,
  "descripcion": "Descripcion pendiente...",
  "capitulos": [
    { "id": 115, "numero": 1, "titulo": "Mirador de Montepinar", "
descripcion": "Los nuevos vecinos llegan con ilusion..." },
    { "id": 116, "numero": 2, "titulo": "Okupas, secuestros...", "
descripcion": "El caos se apodera de las zonas..." }
  ]
},
"costeVisionado": 0.5,
"categoria": "ESTANDAR"
}
```